

# NLP For Social Media Informed Trading

**Mahdi Khemakhem**  
mahdouch@bu.edu

**Ming Hill**  
minghill@bu.edu

## Proposal

### General Idea

In recent years, financial markets have increasingly been influenced by public sentiment and online discussion. Stocks and cryptocurrencies, in particular, have demonstrated volatility driven by viral attention. Looking in the past, notable examples include GameStop(GME) and Dogecoin(DOGE), which surged in value following widespread discussion on platforms such as Reddit and X (previously Twitter). More recently, so-called “meme coins” such as TRUMP and MELANIA have experienced rapid price movement coupled with online hype and excitement. Our project aims to investigate correlations between online mentions of an asset, and its subsequent price movement. Specifically, we want to analyze how different factors, such as source reputation, post traction, post volume, post platform, post topic and more, relate to asset performance. By examining these dynamics, we hope to provide insights into the predictive power of social media activity on financial markets.

### TRUMP Coin Case

TRUMP coin was announced at 9:44 PM on January 17, 2025 (see **Figure 1**). **Figure 2** illustrates the coin’s price surge shortly after the announcement. Initially priced at \$2.50, early adopters could achieve gains of up to 35x, while later stabilization around \$16 still offered an 8x gain. Although this case is atypical, it highlights the potential edge gained by integrating social media insights into trading decisions.

### Project Goal

The goal of our project is to determine whether social media activity can serve as a predictive indicator for stock and cryptocurrency movements, and to define the most impactful signals. To achieve

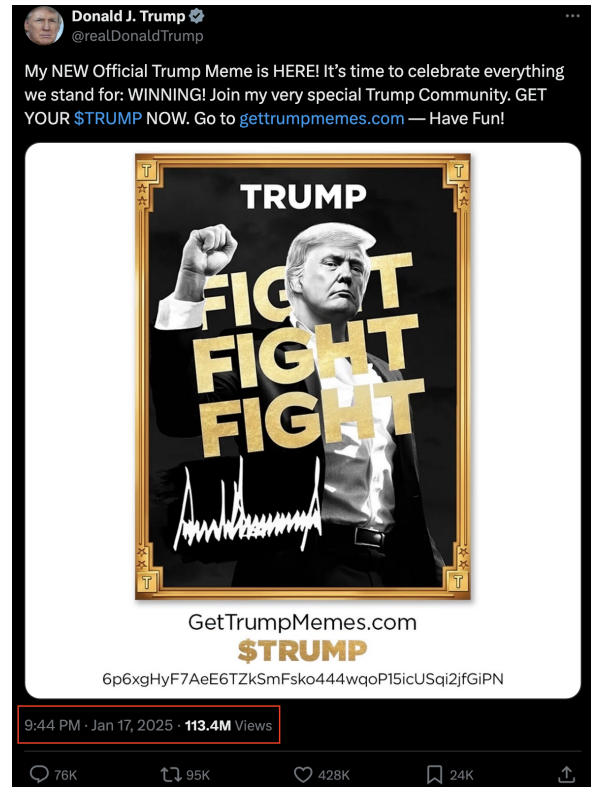


Figure 1: Donald Trump’s official tweet announcing TRUMP coin.

this, we will have a two-fold approach for handling past data, and newly generated data.

### Past Data

For historical data, we hope to use well-established datasets such as [Bitcoin Tweets](#), [crypto-stock-tweets](#), as well as scraping our own data from Reddit and X (formerly Twitter). We want to train a neural network that can generate a holistic score for any given tweet. It is clear that understanding correlations between social media activity, and prices, takes more than sentiment. There are swarms of bots and scammer profiles that constantly bombard social media with fake leads. This is our motivation for the holistic score, which will



Figure 2: Price movement of TRUMP coin following the tweet.

take into account sentiment, but will also work with other features, such as user following to generate a more robust score for rating tweets, and improve our ability to discern true and false positive signals.

### Live Data

With a trained model on past data, we can hope to integrate our model into the processing pipeline for live event streams. Live data adds a different dimension, which is time. With past data, it is hard to get a picture of the evolution of a tweet through time, the traction it receives can happen quickly or slowly, can happen before or after a price surge, and is important for studying. With live data we can integrate our previous holistic model, but add a new layer of tweet performance over-time, and that would necessitate studying this with a recurrent architecture such as recurrent neural networks (RNNs), or long short-term memory networks (LSTMs).

### Inputs/Outputs

At this point, we will have several models that we can build our pipeline with. Models to handle incoming streams and assign learned scores, and models that handle sequences of events.

Ideally, a user will be able to input one or more ticker symbols for the model to continuously monitor, and it will alert the user when it detects the optimal time to purchase an asset.

### Inputs:

- We will have a listener module, that can take specific input (one asset/list of assets) to filter events relevant only to those tickers, or that is passed nothing to indicate that it should listen to everything.
- Real-time data feeds from social media and financial markets that the listener handles.

### Outputs:

- A trading indicator recommending buy or sell actions.
- Analytics including scores and social media traction metrics.

### Project Benefits

If our project is successful, it could provide valuable insights to investors, traders, and financial analysts. Retail investors may use this information to make informed decisions based on online activity, while financial analysts could integrate social media trends into their models. Additionally, our project may highlight the risks of market manipulation through coordinated online discussion through large groups or influential individuals. By understanding these relationships, investors and regulators can better understand the increasingly digital financial landscape.

## Baseline

### Historical Data

We have identified several sentiment analysis datasets with varying scopes (financial, general):

- [TimKoornstra/financial-tweets-sentiment](#)
- [mteb/tweet\\_sentiment\\_extraction](#)
- [StephanAkkerman/crypto-stock-tweets](#)

### Data Preprocessing

At the moment we do not use any preprocessing steps. We are testing the baseline models as is. However, we do plan on experimenting with different preprocessing steps.

### Removals

For tweets, we plan on removing the following:

- URLs
- Mentions
- Hashtags
- Punctuation
- Extra whitespace
- Stopwords

### Edits

We will experiment and consider the following edits:

- Convert to lowercase
- Replace emojis with their corresponding text
  - Example: (☺, smiley face)
- Replace coin and stock tickers with their corresponding text
  - Example 1: (\$GME, ticker)
  - Example 2: (\$GME, GameStop)
- Stemming
- Lemmatization

These edits will be applied to our data depending on the tokenizer/model we use.

### Concatenation

We will be joining our data sources to create a large combined dataset. Our large dataset will

## Models

We have used the following models for our baseline:

- google-bert/bert-base-uncased ([link](#))
- facebook/roberta-base ([link](#))
- cardiffnlp/twitter-roberta-base-sentiment-latest ([link](#))
- mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis ([link](#))
- StephanAkkerman/FinTwitBERT-sentiment ([link](#))

### Evaluation Metrics

For the baseline evaluation of our models, we focused on (TimKoornstra/financial-tweets-sentiment) dataset. We used around 10% of the dataset for evaluation. We used the following metrics:

- Accuracy
- Precision
- Recall
- F1 Score

### Results

Our baseline evaluation revealed varying performance across different models on the financial tweets sentiment dataset. The results are summarized below:

- Twitter RoBERTa achieved the best overall performance with 57.7% accuracy and balanced F1-scores across sentiment classes (negative: 0.52, neutral: 0.56, positive: 0.63).
- DistilRoBERTa-Financial showed consistent performance with 50.6% accuracy and relatively balanced F1-scores (negative: 0.54, neutral: 0.53, positive: 0.45).
- RoBERTa Base showed mixed results with 44.8% accuracy but in reality defaulted to positive sentiment as it was the only class with a non-zero F1 score (0.62).
- BERT Base Uncased achieved 32.1% accuracy, and it had the same issue as RoBERTa Base, defaulting to neutral sentiment as it was the only class with a non-zero F1 score (0.49).

- FinTwitBERT performed poorly with 10% accuracy, suggesting potential implementation issues or dataset mismatch.

We see a decent performance from some of the finetuned models, and very poor performance from the raw pre-trained models. We plan on further finetuning the models on our dataset to see if we can improve the performance.

## Related Works

### CardiffNLP: Twitter RoBERTa

Links: [paper](#), [github](#) The CardiffNLP team developed several models including Twitter-RoBERTa, which is a finetuned version of RoBERTa for Twitter data. They also developed TimeLMs, a GitHub repository containing a collection of language models specifically designed for Twitter data that addresses the temporal aspects of social media content. Their work is particularly relevant to our project, because we are interested in the temporal aspects of tweets and how they relate to stock and cryptocurrency prices.

|               | Emoji | Emotion | Hate | Irony            | Offensive | Sentiment | Stance   | ALL  |
|---------------|-------|---------|------|------------------|-----------|-----------|----------|------|
| SVM           | 29.3  | 64.7    | 36.7 | 61.7             | 52.3      | 62.9      | 67.3     | 53.5 |
| FastText      | 25.8  | 65.2    | 50.6 | 63.1             | 73.4      | 62.9      | 65.4     | 58.1 |
| BLSTM         | 24.7  | 66.0    | 52.6 | 62.8             | 71.7      | 58.3      | 59.4     | 56.5 |
| RoBERTa-Base  | 30.8  | 76.6    | 44.9 | 55.2             | 78.7      | 72.0      | 70.9     | 61.3 |
| TweetEval     | 31.6  | 79.8    | 55.5 | 62.5             | 81.6      | 72.9      | 72.6     | 65.2 |
| BERTweet      | 33.4  | 79.3    | 56.4 | 82.1             | 79.5      | 73.4      | 71.2     | 67.9 |
| TimeLM-19     | 33.4  | 81.0    | 58.1 | 48.0             | 82.4      | 73.2      | 70.7     | 63.8 |
| TimeLM-21     | 34.0  | 80.2    | 55.1 | 64.5             | 82.2      | 73.7      | 72.9     | 66.2 |
| <b>Metric</b> | M-F1  | M-F1    | M-F1 | F <sup>(i)</sup> | M-F1      | M-Rec     | AVG (F1) | TE   |

Figure 3: Table of results for the sentiment task.

When it comes to the sentiment task, they evaluated their models on the TweetEval benchmark which is a unified benchmark for tweet analysis several tasks one of which is sentiment analysis. They achieved 73.7% M-Rec on the sentiment task, see [Figure 6](#).

We plan on finetuning their twitter-roberta model on our dataset to see if we can improve the performance, on finance related tweets.

## Tweet Sentiment Analysis to Predict Stock Market

Links: [paper](#)

In “Tweet Sentiment Analysis to Predict Stock Market” (Palomo 2020), the author proposes a transformer-based pipeline that first classifies tweets into bullish, bearish, or neutral sentiment,

then predicts the next-day direction of a stock’s price. The method can be divided into two parts. First, they fine-tune a FinancialBERT model (initially trained on financial text) on a labeled dataset of finance-related tweets, improving its ability to capture domain-specific jargon. In addition, they pre-process tweets by removing URLs, special characters, and Twitter-specific elements (e.g., hashtags) to further boost classification accuracy. Next, they feed the resulting “weighted sentiment” features—calculated by multiplying each tweet’s sentiment by its retweet count—along with historical stock prices into a Random Forest Classifier to forecast whether a stock’s price will rise or fall.

| Model                                               | Accuracy     | F1           |
|-----------------------------------------------------|--------------|--------------|
| Finetuned FinancialBERT-Sentiment-Analysis          | <b>0.843</b> | <b>0.843</b> |
| FinancialBERT-Sentiment-Analysis                    | 0.803        | 0.803        |
| FinancialBERT-Sentiment-Analysis (no preprocessing) | 0.756        | 0.756        |
| Twitter-roBERTa-base for Sentiment Analysis         | 0.711        | 0.711        |
| BERT                                                | 0.653        | 0.653        |

Figure 4: Table of results sentiment model comparison.

For data, the authors rely on the Twitter-Financial News Sentiment dataset (about 9,938 training and 2,486 validation tweets) to fine-tune the sentiment model, plus historical tweets and stock data for Tesla (TSLA) from August to December 2022. Daily tweet batches are filtered for \$TSLA references and merged with corresponding Yahoo Finance price data. Their results demonstrate that fine-tuning FinancialBERT on domain-specific tweets achieves roughly 84% accuracy in sentiment classification—outperforming generic BERT and roBERTa-based models—while the inclusion of weighted sentiments in the stock prediction model raises overall prediction accuracy to about 91% over the sampled period.

| Model                                             | Accuracy     | F1           |
|---------------------------------------------------|--------------|--------------|
| Random Forest Classifier                          | <b>0.913</b> | <b>0.897</b> |
| Random Forest Classifier (w/o Weighted Sentiment) | 0.869        | 0.846        |

Figure 5: Table of results stock prediction model comparisons.

## Live Data Notes

### Data Sources

The data for this portion is simply a live stream from X.com, with possible addition of other platforms (Reddit, telegram, etc.). We have not found a truly compatible dataset out there for the live data portion. An important feature to have in any tweet

dataset is the timestamp of the tweet release, and the timestamp of the tweet observation. This is what allows us to build an image of how tweet sentiment evolves over time. The generation of this data will therefore be a part of the live data pipeline.

### **Feature Generation**

We plan on integrating the sentiment analysis pipeline trained on the historical data into the live data pipeline. This will allow us to generate features such as the sentiment score, and the sentiment trend over time.

Additionally, we will record the number of likes, retweets, replies to the tweet, as well as the target asset of the tweet, and the price of the asset at the time of the tweet observation.

### **Models**

Using the generated features, we plan on training a model to predict the price movement of the target asset. We plan on using time aware models such as LSTMs, and transformers to hopefully predict the changes in engagement metrics, as well as the price movements of the asset. We may also use statistical models such as Random Forest to classify if we should buy or not.

### **Evaluation Metrics**

Evaluation can be done in a regression fashion, where we predict the price movement of the asset or the engagement metrics, or in a classification fashion, where we predict the direction of the price movement of the asset, or the engagement metrics. We may compare our performance to the performance of the Palomo 2020 paper, in the related works section, as it has a similar target to our live data portion.

## Project Report (Paper)

### Introduction

In recent years, social media has become a major factor in financial markets. Countless examples of assets have seen their prices soar following viral attention on platforms like Reddit and X (formerly Twitter). Though this applies to both stocks (GameStop (GME)) and cryptocurrencies (Dogecoin (DOGE)), the latter has seen a surge in popularity in recent years. Cryptocurrencies, specifically 'meme coins', are especially influenced by social media activity. These assets usually have no real value or backing, and mostly derive their value from social media marketing. Despite this seeming lack of value, these assets can be very lucrative. In fact, according to CoinGecko, the market capitalization of meme coins is estimated to be around \$60 billion dollars (April 2025), and at present forms a legitimate niche for some daytraders.

### Goal

Meme coin markets offer fast paced, extremely volatile trading opportunities with a high risk of scams. Trading these markets must be done with caution, and must be informed by social media activity, due to the lack of real value. This is why we set out to investigate relationships that exist between social media activity and asset behavior. Our two main goals include developing NLP tools that improve the trading experience, as well as analyzing behaviors we observe using these tools, to better characterize meme coin dynamics.

In this project we develop the following models:

- An improved sentiment analysis model specialized for financial tweets, that assigns sentiment scores.
- An engagement model, that predicts tweet engagement metrics.
- A scam classification model that classifies whether a coin is a scam or not based on the coin's code deployed on the blockchain. (Model only available for certain blockchains)

These models enable us to look at the relationship between tweet virality, sentiment, and asset behavior. The scam models are useful in certain blockchains to protect against common exploits. We have also developed a pipeline built on top of these models, that ingests live coins released on the

blockchain, collects related tweets, and monitors their outcomes, allowing us to build a comprehensive dataset of coin behavior that we consequently analyzed.

### Example Input/Output Tweets

**Input:** "This is what winning looks like! \$JPROOF"

**Sentiment Output:** "positive"

**Engagement Output:** 0.989

**Note:** Engagement output is a log of the sum of engagement metrics (favorites, retweets, replies, quotes, bookmarks, views), which should provide better context for the score value shown here.

### Example Input/Output Code

**Input:** [sample code link](#)

**Scam Output:** "scam"

**Note:** If you are interested in the vulnerability in this code it is under the function "permitAllance" which prevents a user from selling the coin, essentially locking their funds.

### Use Cases

There are several use cases where our models may be helpful to people. Some examples include:

- Anomaly Detection: Identifying early promising tokens by looking at tweets with high predicted engagement score (95<sup>th</sup> percentile or more).
- Scam Protection: Provide a layer of security to traders against common vulnerabilities.
- Enhance Trading Bots: Enrich trading decisions using social media sentiment/engagement scores and trends.

### Use Cases: Anomaly Detection

Our engagement model was not trained on the Donald Trump [tweet](#), that announced his coin, but successfully flagged it as an anomaly with a 95.41<sup>th</sup> percentile score. (see script `engagement_distribution.ipynb`)

### Sentiment Analysis

#### Motivation

Understanding the sentiment is critical for predicting meme coin price movements. However, generic sentiment models often struggle and under perform on financial text. This is further exacerbated by short form, slang filled tweets often portrayed in tweets regarding speculative meme

coin assets.

## Existing Approaches

Several existing approaches have attempted to address financial sentiment analysis.

### CardiffNLP: Twitter RoBERTa

Links: [paper](#), [github](#)

The CardiffNLP team developed several models including Twitter-RoBERTa, which is a finetuned version of RoBERTa for Twitter data. They also developed TimeLMs, a GitHub repository containing a collection of language models specifically designed for Twitter data that addresses the temporal aspects of social media content.

|              | Emoji | Emotion | Hate | Irony            | Offensive | Sentiment | Stance   | ALL  |
|--------------|-------|---------|------|------------------|-----------|-----------|----------|------|
| SVM          | 29.3  | 64.7    | 36.7 | 61.7             | 52.3      | 62.9      | 67.3     | 53.5 |
| FastText     | 25.8  | 65.2    | 50.6 | 63.1             | 73.4      | 62.9      | 65.4     | 58.1 |
| BLSTM        | 24.7  | 66.0    | 52.6 | 62.8             | 71.7      | 58.3      | 59.4     | 56.5 |
| RoBERTa-Base | 30.8  | 76.6    | 44.9 | 55.2             | 78.7      | 72.0      | 70.9     | 61.3 |
| TweetEval    | 31.6  | 79.8    | 55.5 | 62.5             | 81.6      | 72.9      | 72.6     | 65.2 |
| BERTweet     | 33.4  | 79.3    | 56.4 | 82.1             | 79.5      | 73.4      | 71.2     | 67.9 |
| TimeLM-19    | 33.4  | 81.0    | 58.1 | 48.0             | 82.4      | 73.2      | 70.7     | 63.8 |
| TimeLM-21    | 34.0  | 80.2    | 55.1 | 64.5             | 82.2      | 73.7      | 72.9     | 66.2 |
| Metric       | M-F1  | M-F1    | M-F1 | F <sup>(t)</sup> | M-F1      | M-Rec     | AVG (F1) | TE   |

Figure 6: Table of results for the sentiment task.

When it comes to the sentiment task, they evaluated their models on the TweetEval benchmark which is a unified benchmark for tweet analysis several tasks one of which is sentiment analysis. They achieved 73.7% M-Rec on the sentiment task, see **Figure 6**.

### Tweet Sentiment Analysis to Predict Stock Market

Links: [paper](#)

In “Tweet Sentiment Analysis to Predict Stock Market” (Palomo 2020), the author proposes a transformer-based pipeline that first classifies tweets into bullish, bearish, or neutral sentiment, then predicts the next-day direction of a stock’s price. The method can be divided into two parts. First, they fine-tune a FinancialBERT model (initially trained on financial text) on a labeled dataset of finance-related tweets, improving its ability to capture domain-specific jargon. In addition, they pre-process tweets by removing URLs, special characters, and Twitter-specific elements (e.g., hashtags) to further boost classification accuracy. Next, they feed the resulting “weighted sentiment” features—calculated by multiplying each tweet’s

sentiment by its retweet count—along with historical stock prices into a Random Forest Classifier to forecast whether a stock’s price will rise or fall.

For data, the authors rely on the Twitter-Financial News Sentiment dataset (about 9,938 training and 2,486 validation tweets) to fine-tune the sentiment model, plus historical tweets and stock data for Tesla (TSLA) from August to December 2022. Daily tweet batches are filtered for \$TSLA references and merged with corresponding Yahoo Finance price data. Their results demonstrate that fine-tuning FinancialBERT on domain-specific tweets achieves roughly 84% accuracy in sentiment classification—outperforming generic BERT and RoBERTa-based models—while the inclusion of weighted sentiments in the stock prediction model raises overall prediction accuracy to about 91% over the sampled period.

### Sentiment Training Pipeline

To address the unique challenges on meme-coin sentiment prediction, we fine-tuned CardiffNLP’s Twitter-RoBERTa model, on a large dataset of labeled financial twitter data. The tweets were annotated as positive, negative or neutral sentiment. To build a sentiment model for meme coin-related tweets, we performed the following steps:

- Calculate baseline metrics
- Compiled historical financial tweet data with sentiment annotations from two datasets
- Preprocessed the data
- Fine-tuned the CardiffNLP Twitter-RoBERTa model on the processed dataset.

### Metrics

To evaluate our baseline and sentiment analysis model, we primarily used the **F1 score** and **accuracy**, while also reporting **precision**, **recall** for our model.

The F1 score is the harmonic mean of precision and recall, making it especially valuable in scenarios where class imbalance is present. In sentiment analysis—particularly with financial tweets—there is often a disproportionate number of positive or neutral tweets compared to negative ones. Relying solely on accuracy can be misleading in such cases, as a model might achieve high accuracy by predominantly predicting the majority class.

By contrast, the F1 score balances both false positives and false negatives, providing a more reliable

measure of how well the model performs across all sentiment classes. This is particularly important in financial contexts, where misclassifying a negative tweet as positive can lead to incorrect trading decisions.

### Baseline

For our baseline metrics, we use 5 different models.

- google-bert/bert-base-uncased
- facebook/roberta-base
- cardiffnlp/twitter-roberta-base-sentiment-latest
- mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis
- StephanAkerman/FinTwitBERT-sentiment

All baseline models were evaluated on a 10% held-out portion of Tim Koornstra’s Financial Tweet Sentiment dataset. We recorded each model’s **accuracy** and **weighted F1 score**, as summarized in **Table 1**.

### Data

We used two publicly available datasets of financial tweets to train our sentiment analysis pipeline. The first is the **Tim Koornstra Financial Tweet Dataset**, which contains over 38,000 annotated tweets related to finance. The second is the **Abdur Rahman’s Twitter Financial News Sentiment Dataset** from Kaggle, which includes over 11,900 labeled tweets.

Both datasets contain individual tweets paired with sentiment labels. While neither dataset focuses specifically on meme coin-related tweets, the language and tone used in discussions about financial assets like stocks were sufficiently similar to provide a strong foundation for fine-tuning a domain-adapted sentiment model.

### Data Processing

With both datasets combined, we obtained over 50,000 labeled instances for sentiment model training. To prepare the data, we first standardized the column names and sentiment labels to ensure consistency across both datasets. Sentiment values were mapped to a unified three-class format (positive (2), negative (1) and neutral (0)).

After standardization, we merged the datasets into a single corpus and split it into training (80%), validation (10%), and test (10%) sets for model development and evaluation.

### Preprocessing

All tweets were preprocessed prior to training. We replaced Twitter usernames with the placeholder token @user and URLs with the token http. This step reduced noise and improved generalization by minimizing variability due to user-specific content or external links.

### Training/Finetuning

To develop a sentiment classifier tailored for meme coins we fine-tuned the cardiffnlp/twitter-roberta-base-sentiment-latest model using our domain-specific labeled data.

### Training Setup

Fine-tuning was performed using the Hugging Face transformers library and PyTorch. The model was trained for 40 epochs using the AdamW optimizer with a learning rate of  $2 \times 10^{-5}$  and a weight decay of 0.01. A linear learning rate scheduler with no warm-up steps was applied. Each training batch contained 32 examples, and gradient clipping with a max norm of 1.0 was used to prevent gradient explosion.

### Evaluation Metrics

Model performance was evaluated after each epoch on a validation set using accuracy, precision, recall, weight F1 score and average loss.

Evaluation was conducted using a custom function that calculated metrics based on predictions from a batched DataLoader. Given the class imbalance in the dataset, the weighted F1 score served as our primary performance metric.

### Model Selection

At the end of each epoch, if the validation accuracy improved relative to previous epochs, the model and tokenizer were saved to disk as the current best model. Final evaluation was then conducted using a separate test set to assess generalization performance.

### Results

The fine-tuned model achieved strong performance across all evaluation metrics on the test set.

- **Accuracy:** 0.8949
- **Precision:** 0.8945
- **Recall:** 0.8949
- **F1 Score (weighted):** 0.8946

| Model                             | Accuracy     | F1 Score      |
|-----------------------------------|--------------|---------------|
| <b>Fine-tuned Twitter-RoBERTa</b> | <b>89.5%</b> | <b>0.8946</b> |
| Twitter-RoBERTa                   | 57.7%        | 0.57          |
| DistilRoBERTa-Financial           | 50.6%        | 0.56          |
| RoBERTa Base                      | 44.8%        | 0.62          |
| BERT Base Uncased                 | 32.1%        | 0.49          |
| FinTwitBERT                       | 10.0%        | –             |

Table 1: Performance comparison of sentiment classification models.

- **Average Loss:** 0.9233

These values indicate that the model is both consistent and balanced in its predictions across sentiment classes. The high F1 score, in particular, demonstrates that the model performs well even in the presence of class imbalance, making it suitable for real-world financial tweet classification. The average loss suggests good convergence and stable optimization during training.

### Comparison to Baseline Models

Our fine-tuned Twitter-RoBERTa model significantly outperformed all baseline models, achieving an accuracy of 89.5% and a weighted F1 score of 0.8946 which can be observed in **Table 1**. In contrast, the best-performing baseline model—Twitter-RoBERTa without additional fine-tuning—achieved only 57.7% accuracy and an F1 score of 0.57. Other models such as DistilRoBERTa-Financial and RoBERTa Base performed moderately (accuracy below 51%), while BERT Base and FinTwitBERT underperformed, with accuracies of 32.1% and 10.0%, respectively.

While our metric shows a significant improvement over baseline models, the gain is comparatively modest when evaluated against previously published approaches. For instance, the fine-tuned FinancialBERT model achieved an F1 score of 0.843 (**Figure 4**). This improvement highlights the effectiveness of using a Twitter-specific architecture combined with domain-specific fine-tuning and preprocessing tailored to the informal language used in financial social media.

The performance gap between our model and the baselines can be attributed to two key factors. First, our model was fine-tuned on over 50,000 labeled financial tweets, allowing it to learn sentiment cues and slang specific to the financial discourse on Twitter. Second, we applied targeted preprocessing to clean domain-specific noise—such as usernames, hashtags, and URLs—that generic models often overlook.

In contrast, the baseline models lacked either financial domain alignment, Twitter-specific context, or both. For example, general-purpose models like BERT Base and RoBERTa Base were trained on non-financial, formal text corpora, while models like DistilRoBERTa-Financial and FinTwitBERT were either trained on financial data from non-social media sources or lacked Twitter-specific adaptation. As a result, these models struggled to generalize to the informal, fast-evolving language used in meme coin discussions on Twitter. Our results underscore the importance of domain adaptation—both in content (financial) and platform (Twitter)—when applying NLP models in specialized contexts.

### Engagement Prediction

The engagement prediction model portion of our work aimed to predict a real value for a holistic engagement score, aggregating favorites, retweets, replies, quotes, bookmarks, and views, in hopes of capturing the virality of a tweet. We treated this as a regression problem to generate the following:

$$E(\text{score}|\text{tweet features})$$

There were several challenges associated with building this model:

- **Lack of data:** We were unable to find datasets that linked tweets to their engagement metrics
- **Lack of baseline models:** We were unable to find models that tackled our specific needs
- **Twitter API access is extremely limited** for free use, and very expensive paid tiers (\$100, \$5000)/month

**Data Requirements:** We did not only require datasets with tweets and snapshots of their engagement metrics. We also needed information about when those snapshots were taken, relative to the creation date of a tweet. There were some datasets such as [stock\\_market\\_tweets](#), that did

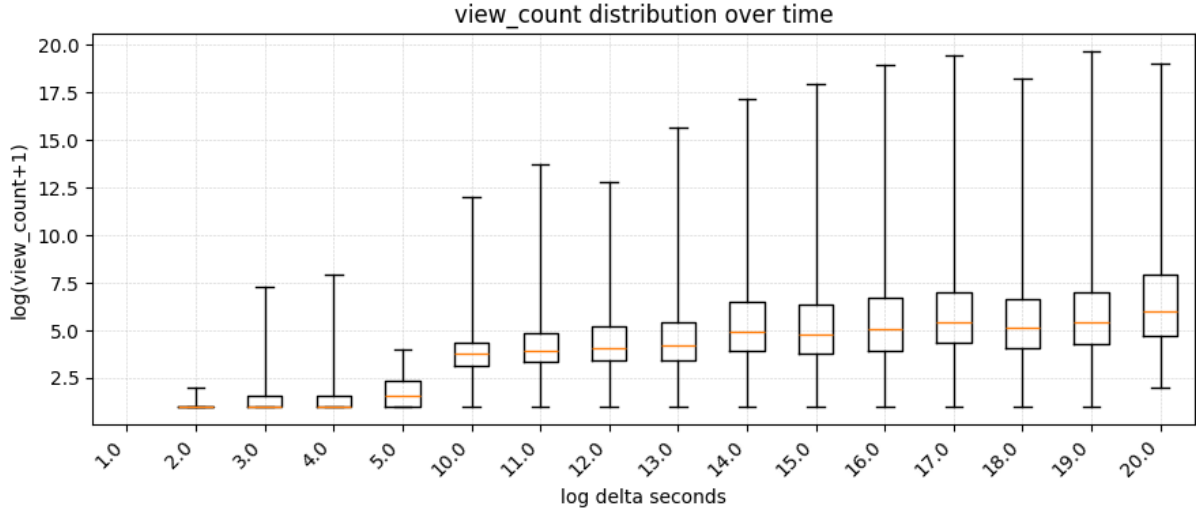


Figure 7: View count distribution overtime (log-log).

---

**Algorithm 1** Process A — Real-time Tweet Ingestion

---

**Require:** event stream  $S$

- 1: **init** empty FIFO queue  $Q$
  - 2: **for all** tweet  $t \in S$  as it arrives **in parallel do**
  - 3:   enqueue  $t$  into  $Q$
  - 4: **end for**
- 

collect some engagement metrics, but lacked time information. **Figure 7** shows view count behavior over time. The shape is reminiscent of a sigmoid, which makes sense when thinking about the life-cycle of a tweet, eventually the engagement will level off, and the tweet will be "dead". We will be assuming this sigmoid behavior when dealing with engagement metrics.

**Collection Method:** With the sigmoid shape in mind, we decided on **Algorithm 1** for tweet scraping and **Algorithm 2** for token monitoring.

One immediate concern with this algorithm, is that as the size of the queue grows, the time between monitoring observations also grows. This is OK, because we are looking for late-stage tweet observations that have essentially stabilized and will not gain more sizeable traction.

**X/Twitter Scraping:** We used twikit to get around X's API restrictions. Twikit masquerades as the official web client allowing free but heavily rate-limited scraping. Some techniques we employed for smooth scraping experience:

- Account rotations: We made several fake accounts and constantly rotated them to avoid

---

**Algorithm 2** Process B — Engagement Monitoring Worker

---

**Require:** shared queue  $Q$ , early stopping threshold  $n$

- 1: **while**  $Q$  is not empty **do**
  - 2:    $t \leftarrow \text{DEQUEUE}(Q)$
  - 3:    $m_{\text{new}} \leftarrow \text{GETENGAGEMENTMETRICS}(t)$
  - 4:   **if**  $m_{\text{new}}$  unchanged for last  $n$  checks **then**
  - 5:      $\text{MARKDONE}(t)$
  - 6:   **else**
  - 7:     enqueue  $t$  back into  $Q$
  - 8:   **end if**
  - 9: **end while**
- 

bans as much as possible

- Rate Limit Tracking: We followed the recommended rate-limits for client endpoints strictly
- Random data collection: Our scripts had several random sleeping periods to try and reduce detection.

Following this we were able to collect data for 60K tweets over a two week period.

**Data Pre-processing:** Before we trained the model, we did some pre-processing on our data to construct feature vectors and their associated target values. Our most recent preprocessing pipeline (see preprocess.py), handles time features, text features, and computes the target value. Some notable decisions that we made:

- tweet age (seconds) transformation (log/sig-

moid) for stable predictions on mature tweets. Since this value can get quite large, it was essential to apply these transformations to stabilize predictions, as well as capture information regarding the maturity of a tweet.

- text input embeddings are generated using "all-MiniLM-L6-v2" embedding model from sentence-transformers. We also used a tf-idf embedder in training which did comparably well, but stuck to the sentence-transformers for the final model.
- target is computed as the log of the sum of favorites, retweets, replies, quotes, bookmarks, and views.

| Dataset    | MSE    | RMSE   | R <sup>2</sup> |
|------------|--------|--------|----------------|
| Train      | 0.1558 | 0.3947 | 0.8413         |
| Validation | 0.1759 | 0.4194 | 0.8202         |
| Test       | 0.1690 | 0.4111 | 0.8232         |

Table 2: XGBoost model metrics

**Model Training:** For training we used an XGBoost regressor as it did better than other light weight models in early training rounds (lightgbm, random forest regressor, catboost), and we saw marginal benefits from finetuning transformer based architectures like roBERTa (both base, and the finetuned model from CardiffNLPO), at the cost of long training times. The fast training and decent performance level of xgboost were the main reasons for picking it. See **Table 2** for training metrics.

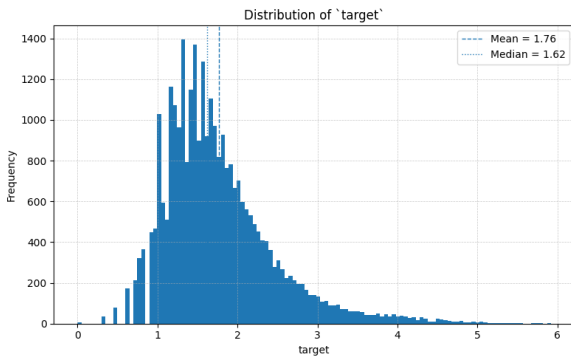


Figure 8: Target variable distribution in engagement prediction task.

**Resulting Target Distribution:** **Figure 8**, and

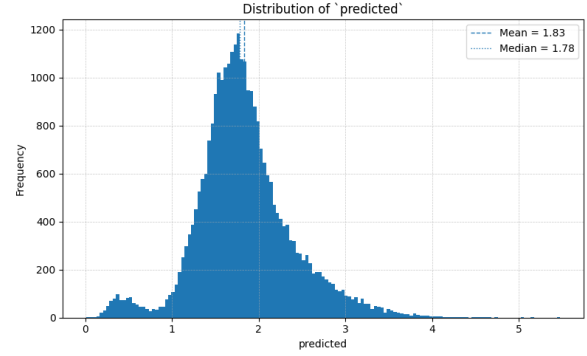


Figure 9: Predicted Target variable distribution in engagement prediction task.

### Algorithm 3 Dataset Building

**Require:** event stream  $S$

```

1: init empty FIFO queue  $Q$ 
2: for all tweet  $t \in S$  as it arrives in parallel do
3:   buy small token amount ( $< 0.001\$$ )
4:   wait 5 minutes
5:   attempt sell
6:   if sell_success then
7:     record 1
8:   else
9:     record 0
10:  end if
11: end for

```

**Figure 9** show the distribution of the original target variable, and a distribution of model predictions. Overall the model learned a good representation of the target variable.

### Scam Classification

For tackling the scam classification we are hoping to train a model on the source code of tokens published on the blockchain, to learn patterns between the code used and whether or not it is a scam.

$$P(\text{scam}|\text{source code})$$

**Collection Method:** We built our own simple labeled dataset for this task, that contains smart contract code, and an associated label for the ability to sell. The collection process is depicted in **Algorithm 3**.

**Data Pre-processing:** In order to build our feature vectors we used a combination of TF-IDF to generate document embeddings, and one-hot encodings for presence of common functions in solidity smart contracts. No changes were done to the binary label (1 can sell/no scam, 0 cannot sell/scam).

| Class        | Precision    | Recall | F <sub>1</sub> |
|--------------|--------------|--------|----------------|
| 0            | 0.44         | 0.94   | 0.60           |
| 1            | 0.93         | 0.37   | 0.53           |
| Accuracy     | 0.57 (n=102) |        |                |
| Macro avg    | 0.68         | 0.66   | 0.57           |
| Weighted avg | 0.76         | 0.57   | 0.56           |

Table 3: Validation-set scam classification report

| Class        | Precision    | Recall | F <sub>1</sub> |
|--------------|--------------|--------|----------------|
| 0            | 0.42         | 1.00   | 0.59           |
| 1            | 1.00         | 0.27   | 0.42           |
| Accuracy     | 0.52 (n=102) |        |                |
| Macro avg    | 0.71         | 0.63   | 0.51           |
| Weighted avg | 0.80         | 0.52   | 0.48           |

Table 4: Test-set scam classification report

**Model Training:** We simply used an xgboost classifier model, again because it was light, and training was very fast. Additionally, it outperformed other models provided by the sklearn suite. Metrics shown in **Table 3** and **Table 4**. Though the accuracy is not the best, the recall is the important value to pay attention to in the case of scam detection since a mislabel of a scam is more serious than a mislabel of a safe token, and our model shows very high recall in the scam case (0).

**Baseline Comparison:** Existing approaches for detecting scams, specifically honeypots-which are tokens with vulnerabilities in them that prevent you from selling and cashing in your profits-, usually involve simulating transactions (read more [here](#)), and websites/apis are available as a service to do this on your behalf (Example Service: [link](#)). However this baseline is not very good. I ran the service on our own dataset, results shown in **Table 5**, and even though accuracy is comparable, the online service is actually very bad at spotting scams (scam recall 0.11). This poor performance is because online services only account for vulnerabilities that prevent simulated buys and sells, but do not account for sophisticated vulnerabilities that activate post-buy. We will not discuss the exact mechanisms here but the following is an [article](#) that should provide a good idea of how these scams operate.

| Class        | Precision    | Recall | F <sub>1</sub> |
|--------------|--------------|--------|----------------|
| 0            | 0.27         | 0.11   | 0.16           |
| 1            | 0.63         | 0.83   | 0.72           |
| Accuracy     | 0.58 (n=102) |        |                |
| Macro avg    | 0.45         | 0.47   | 0.44           |
| Weighted avg | 0.50         | 0.58   | 0.52           |

Table 5: Honeypot.is scam classification report

## Pipeline

We build a pipeline to tie all of our work together, and help us study price and social media activity dynamics. The pipeline is depicted in **Figure 10**. The workflow is as follows:

- Listen to new token releases on the blockchain
- Optional: Apply scam filter. Not used here as Solana does not open source on-chain smart contract code
- Wait for a token to die, either through a price death ( $< 95\%$  of original price), or through a rugpull (fund theft scam conducted by the creator of the token).
- Gather tweets that mention the asset.
- Analyze results

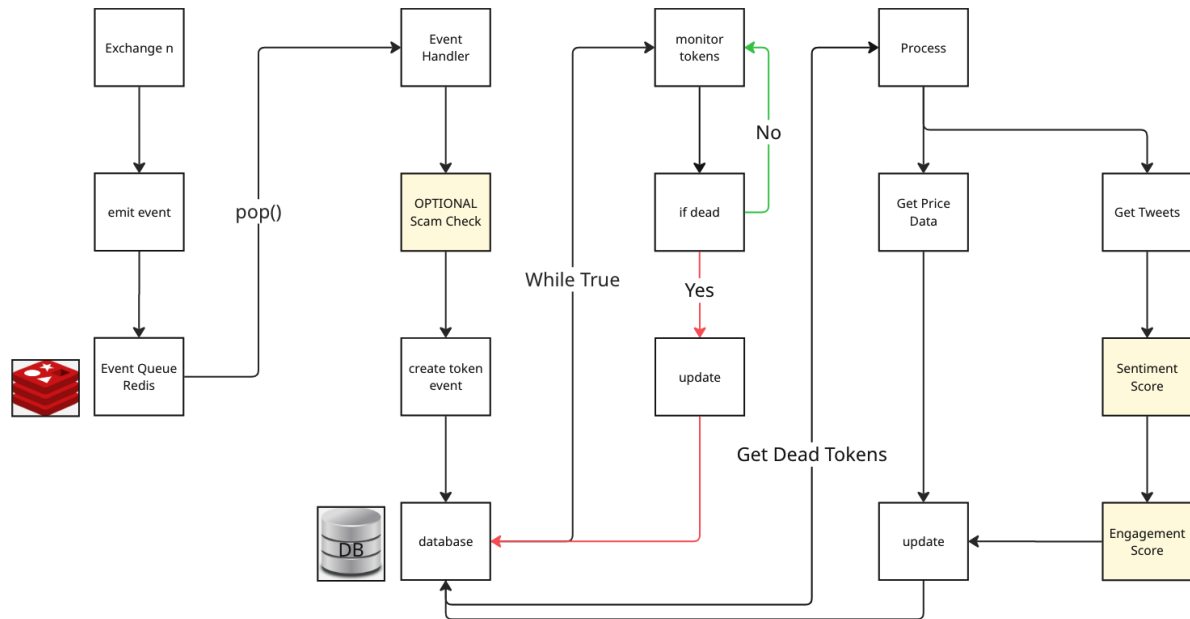
Overall we received around 4000 tokens, and 1000 tweets.

## Data Sources

We collect token data by subscribing to token creation events on the blockchain, specifically Solana. These events streams are publicly available, and provide metadata information that will be used later to track a coin/token in its life cycle. Solana endpoints are also used to collect transaction data which helps us identify rugpulls. We collect price data using CoinGecko terminal API, which provides crypto price data. We collect tweet data using the same approach mentioned in the Engagement section, using twikit.

## Coin Lifecycle

It is important to realize that most of the tokens we are dealing with are scams given the unregulated nature of the crypto space. It is rare to find a coin that is backing a serious project, and ownership of the coin represents real equity in the project, this makes sense because such projects take months



if not years to build and deploy. It is safe to assume that every single coin release you observe is a scheme of some sorts. These schemes differ in complexity and strategy:

- **No-marketing:** In this strategy, scammers play a numbers game, and mass release coins to trap the occasional trading bot or average user. These tend to be short-lived with a lifespan of minutes if not seconds.
- **Marketing:** In this approach, scammers broadcast their tokens on various social channels (X.com, telegram, reddit), in hopes of trapping a larger volume of traders. These schemes tend to have longer lifespans which vary depending on the the scamming entities (profit thresholds, time based scheme, etc..), and if properly orchestrated may even last months, but usually last hours to days.

Regardless of the strategy used, these scams usually share the same appearance in terms of price behavior, an initial rise followed by sudden or gradual fall, this is because they lack any intrinsic value, and cannot self-sustain. **Figure 11** shows an example coin behavior. The sharp price drops, represent mass owner sell offs to cash in their profits. **Figure 12** shows a different example, where the funds are stolen. The yellow transaction represents fund removal by the owner of the coin which includes traders who invested. Our pipeline is equipped

with the ability to detect both outcomes, and **Figure 13** shows the outcome split at the end of data collection phase.



Figure 11: Example coin lifecycle.

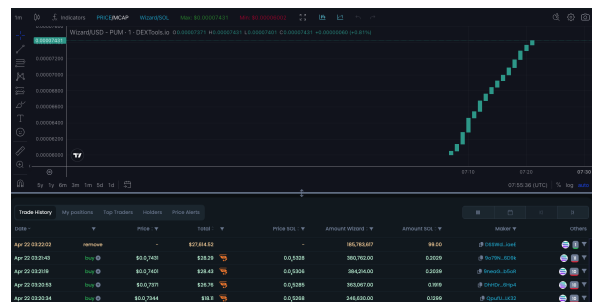


Figure 12: Example coin rugpull.

## Coin Investigation

Once a coin is marked dead, it moves to the investigation stage in our pipeline, where we collect info such as:

- Duration: How long a coin lived

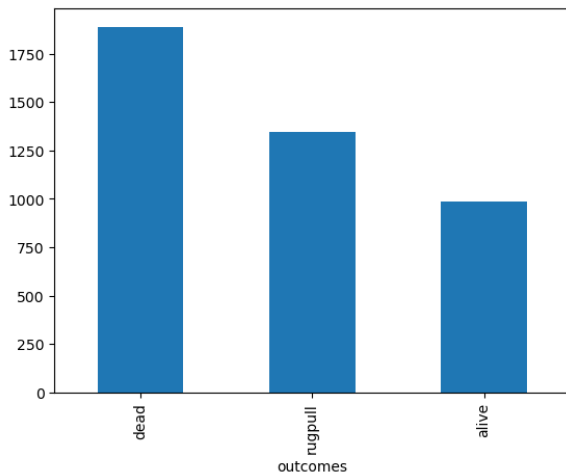


Figure 13: Coin outcome split at the end of collection period.

- All time high time: Where the highest price occurred relative to the creation timestamp of the coin
- Tweets: All the tweets that mention the coin's addresses
- Scammer twitter account: Out of all the tweets we attempt to resolve which ones belonged to the scammers running the marketing scheme

This part of the pipeline is the least consistent one, as our tweet search is bound to coin address mention. We cannot use coin tickers because they are not unique on Solana. You can have multiple coins with the same name but different addresses. For our scammer account detection, we employ two strategies:

- Look through Solana metadata. Coins are sometimes tagged with socials information upon release.
- Look for the coin address in the twitter user's description. We cannot look through their tweets since they can mention other addresses that would confuse ownership.

**Figure 14** shows the pattern we are looking for to resolve users. This two step process ensures we correctly assign user tweets to the correct coin in our database.

One interesting challenge that we did not think of prior to building the pipeline, is account deletion. We witnessed teams that deleted their associated twitter accounts as soon as their scam

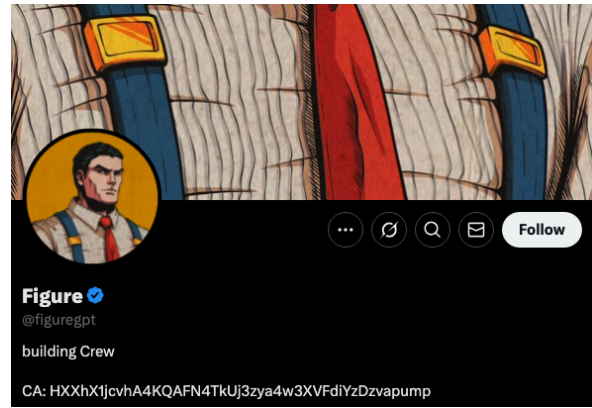


Figure 14: Example marketing account.

operation ended, leading to information loss. This is clever by scammers trying to protect their operation, and would warrant a redesign of the pipeline as future steps, where tweets are actively monitored rather than waiting for a coin to die before searching tweets.

## Results/Findings

We took a look at how tweet presence affects the timing of rugpulls and all time highs. **Figure 15** shows the distribution of log time deltas for coins with tweets and no tweets. The tweet group had a mean of 2.366, and the no-tweet group had a mean of 2.290, which translates to a 40 second difference. This was not the most meaningful difference, and is likely due to the current tweet collection implementation, which lack the necessary fidelity. A more interesting observation is shown in **Figure 16**, where we see that coins with tweets have a higher minimum rugpull time (100 seconds). Additionally, the tweet group had a mean of 2.917, and the no-tweet group had a mean of 2.790, which translates to a 210 second difference. Though this is not conclusive it definitely supports the idea that coins that get marketed will last longer. More data, and more reliable pipelines are crucial future steps to verify this. An unexpected addition to our suite of scam detection approaches, actually came from this pipeline. We looked at common coin creator addresses which released multiple tokens, and it turns out there are some addresses that are very active and are creating multiple tokens, see **Figure 17**, each bar is a unique creator. This opens up interesting possibilities for scam detection and all time high predictions, by profiling creators in order to reflect their marketing/scamming behaviors.

$$P(\text{scam}|\text{creator})$$

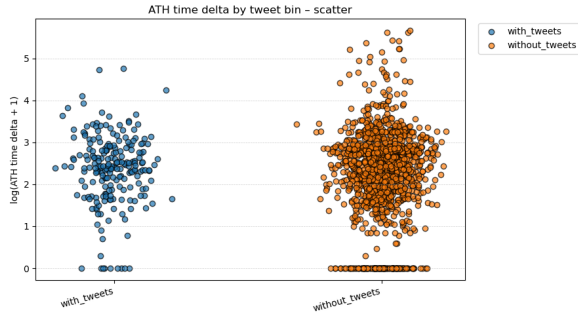


Figure 15: Time scatter for tweet vs no tweet (ATH).

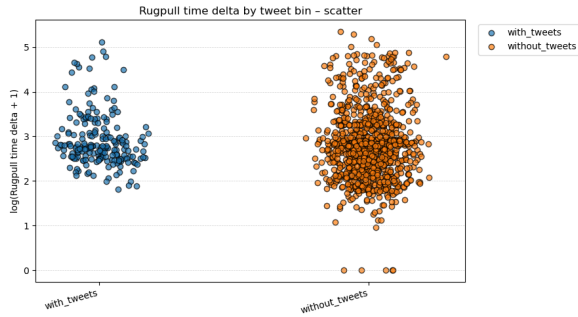


Figure 16: Time scatter for tweet vs no tweet (Rugpull).

$$E(\text{ATH}|\text{creator})$$

**Figure 18**, already shows some differences in dynamics of different tokens generated by different unique creators.

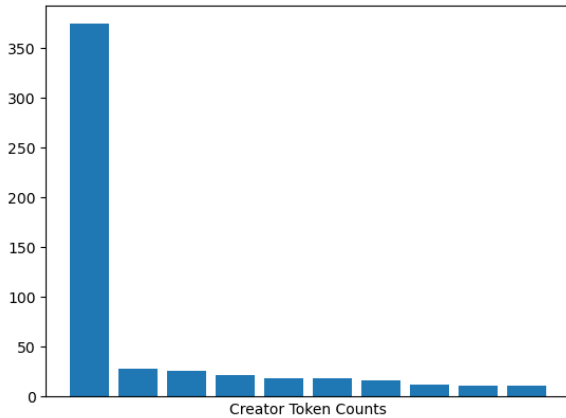


Figure 17: Creator counts.

We also looked at how the engagement and sentiment scores might relate to the ATH and rugpull time. We scattered the average sentiment score for tweets of a coin, vs their ATH and rugpull time deltas/gains. The only meaningful effect we observed is shown in **Figure 18** which shows a slightly positive correlation between average sentiment score, and price gain (ATH

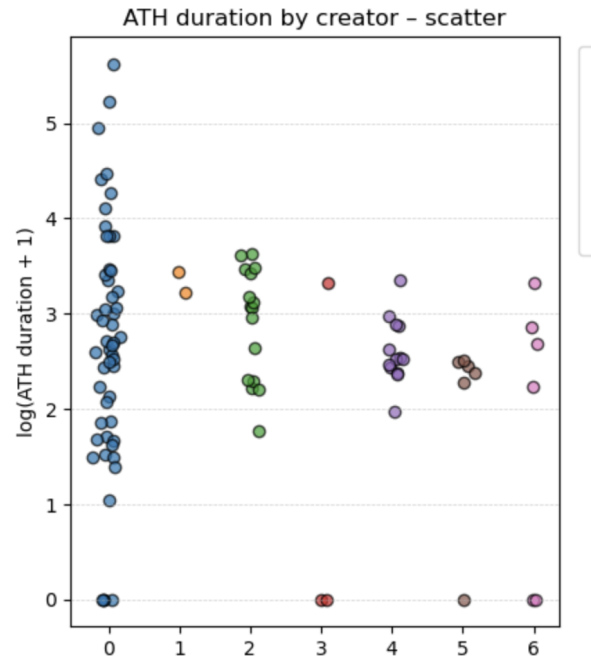


Figure 18: Profiling creator behavior

peak). No other correlations were found. We believe again that this is due to the current data collection pipeline, and it would be interesting to further study these relationships with improved data collection.

Finally, we ran some case studies on certain

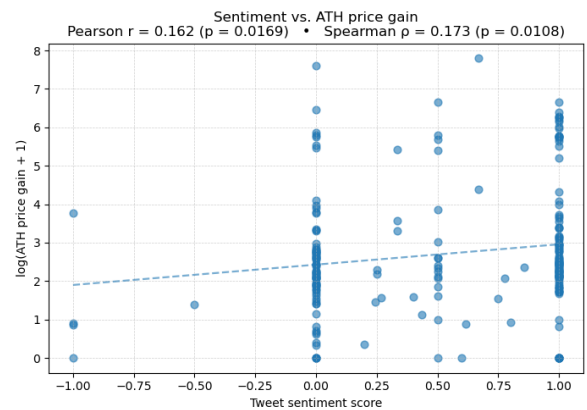


Figure 19: Sentiment vs. ATH price gain

addresses and found some really interesting events, like this token (2iT1EMtMwn8jj3fPDDXzxKbgi-owbm8JQahd2EXeYpump), shown in **Figure 20**. It clearly shows the start of the marketing campaign (blue dots) around the same time the token is created. Early tweets are more dense, and the ATH occurs around the same dense region. This is followed by a reduction in marketing effort. This

campaign in particular was successful, as it did not die/get rugpulled quickly. Though we did not have the chance to build an LSTM model to handle sequence data. A case like this is a great example to highlight the possibilities for future models.

## Conclusion

In this project we built the following:

- Sentiment model
- Engagement model
- Scam model
- A pipeline that combines all of the models

We successfully improved sentiment model performance in the context of crypto/finance tweets, we created a new model for predicting engagement with tweets, and we improved recall performance of currently available scam detection solutions. We learned so many things on this project, some notable things include:

- New scraping methods for getting data from X
- Tweet engagement behavior
- Function signatures + `tf_idf` as a powerful embedding for scam detection
- New scammer behaviors! Such as twitter account cleanup after a scam is completed
- Possible future avenues such as scammer profiling, and updating data collection pipeline

This was an extremely enriching project with a lot of depth in different areas, and we are looking forward to working more on this even beyond the scope of this class.

## Appendix (Replicability)

We would recommend downloading a larger zip file that includes all the data/models pre-loaded from here. It is around 600Mb since it contains a torch model, but otherwise should save a lot of time in terms of getting things up and running. ([google drive link](#))

## Sentiment

For downloading data, follow these steps:

- To download the TimKoornstra/financial-tweets-sentiment dataset, uncomment and run cell 2 of `sentiment/data_process.ipynb`.
- To download the borhanitrash/Twitter Financial News Sentiment Dataset, visit [this link](#) and download the `sent_train.csv` and `sent_valid.csv` files.
- Place the downloaded datasets in the same directory as `data_process.ipynb`, then run all cells in the notebook.

To train the model, open `sentiment_analysis.ipynb` and navigate to the `main()` function. Set the variable `mode` to "train" to train a new model, or to "eval" to evaluate it on a test set.

## Engagement

Engagement code is two fold. The live data collection portion, and the model training portion.

In order to run data collection:

- populate twitter account details (username, password, email), I already provided some fake accounts we used.
- initialize account server
- run `new_tweets.py`
- in a separate terminal run `monitor.py`

In order to run training:

- download requirements
- turn `tweets.db` into csv
- update config to point to csv file
- run `preprocess.py`
- run `train.py`

## Scam Detection

We would not recommend running the scam pipeline because it performs actual blockchain transactions. You would need to do the following:

- `INFURA_API_KEY`: get an infura [api key](#)
- `MNEMONIC`: enter your wallet mnemonic

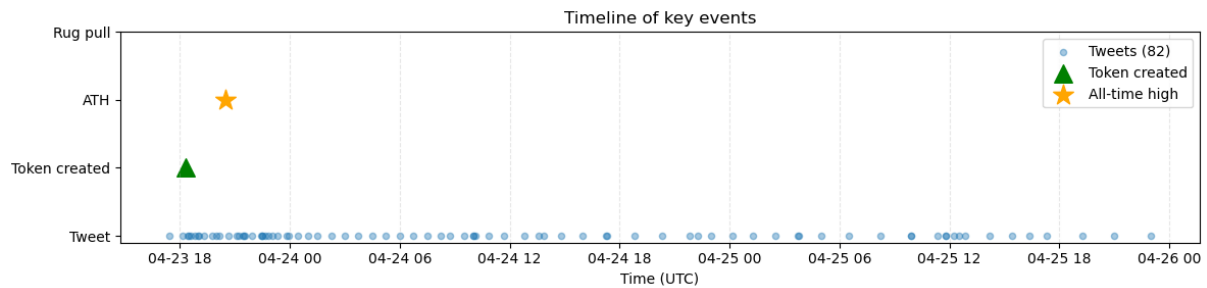


Figure 20: Case

- BASE\_SCAN\_API\_KEY: get a base scan [api key](#)
- You MIGHT need an OPENAI\_API\_KEY, but this is a secondary feature and can be disabled
- download requirements
- run redis-server
- run event\_fetcher.py
- run single\_worker.py

### Pipeline-collection

In order to run this workflow do the following:

- download requirements
- set blockchain endpoints of choice
- initialize account server
- run init\_listeners.py
- run handler.py
- run monitor.py
- run process\_ath.py
- run process\_rugged.py

This will slowly build an sqlite3 db with multiple tables.

### Pipeline-analysis

Marketing.ipynb and case\_study.ipynb are the main notebooks to look at here. Again we recommend getting the data directly from the google drive link, since reconstructing the data would be very difficult.